

INT202W09_最小生成树，流网络，二分匹配

最小生成树 Minimum Spanning Tree(MST)

图G的最小生成树MST满足：

MST包含G的所有顶点

MST不包含环（是树）

在所有可能的生成树中，MST的边权重之和最小

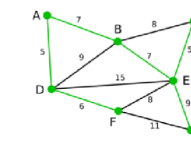
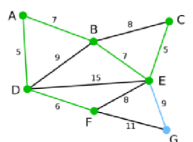
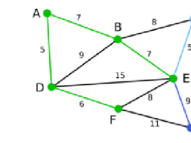
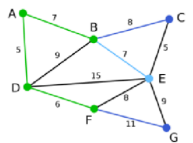
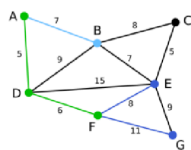
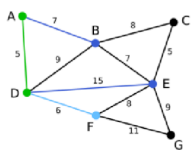
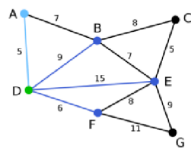
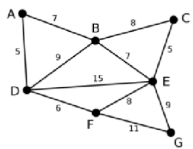
普利姆算法 Prim's algorithm

寻找MST的贪婪算法

从一个根顶点构建MST的方法

在每一步，寻找不在树中且最小化开销且不成环的边

1. 从一个顶点开始，将其他顶点添加到生成树中，直到包含所有节点
2. 避免循环：只选择连接当前树的顶点（集合U）到外部节点（集合V-U）的边
3. 最小化边权重和：每次只选择连接树和非树顶点的最短边（贪心）



克鲁斯卡尔算法 Kruskal's Algorithm

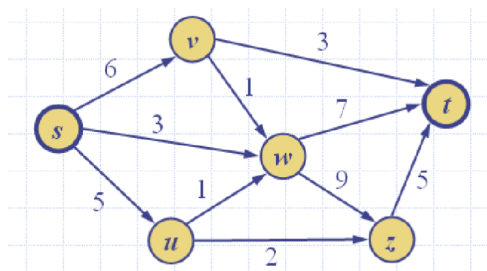
每次选择全局最短的边，如果成环则拒绝，直到所有顶点连接。

■ 流网络 flow network

对流网络N包含：

一个非负整数权重的加权图G，每边权重 $c(e)$ 称为容量（capacity）e

两个顶点s（source，源）和t（sink，汇），s没有入度，t没有出度



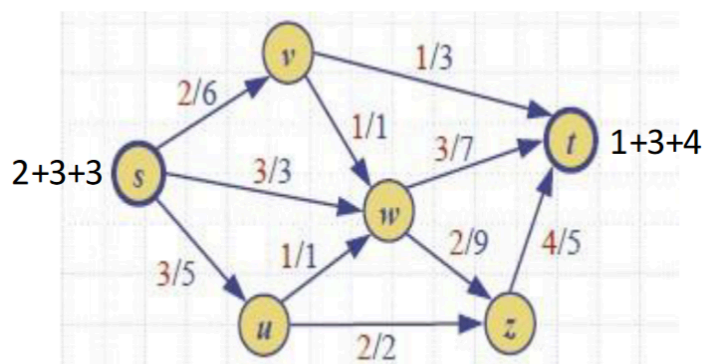
■ 流 flow

网络N的流f是将整数值 $f(e)$ 分配给满足以下属性的每个边e：

- Capacity Rule:对每个边e, $0 \leq f(e) \leq c(e)$
- Conservation Rule:对每个不是起点终点的顶点有 $\sum_{e \in E^-(v)} f(e) = \sum_{e \in E^+(v)} f(e)$; 其中 $E^-(v)$ 和 $E^+(v)$ 代表一个顶点的进入和离开边

流的值记作 $|f|$ ，是来自源的总量=进入汇的总量

最大流问题：找到给定网络 N 的最大流

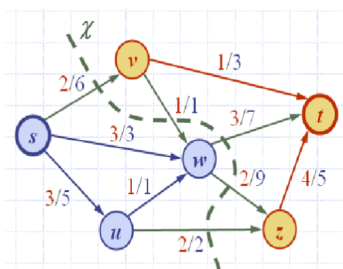


|| Cut割：

一个割可以把网络N分为两个顶点集 $X = (V_s, V_t)$, $s \in V_s, t \in V_t$

Forward edge of cut X: 从 V_s 射入 V_t

Backward edge of cut X:从 V_t 射入 V_s

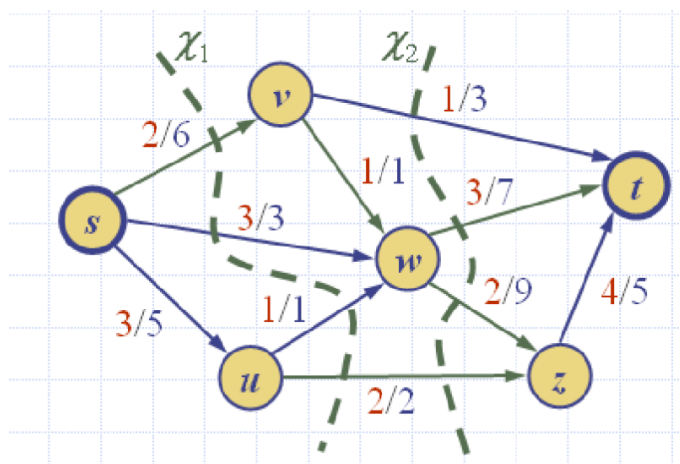


$V_s = \{s, u, w\}$
 $V_t = \{v, t, z\}$
 Forward edge: $\{(s,v), (w,t), (w,z), (u,z)\}$
 Backward edge: $\{(v,w)\}$

计算流/容量

穿过 (across) 割X的流 $f(X)$: Forward edge的总流量-Backward edge的总流量

割X的容量 $c(X)$: Forward edge的总容量



The total flow/capacity of the cut are

- ▷ $c(X_1) = 12 = 6 + 3 + 1 + 2$
- ▷ $c(X_2) = 21 = 3 + 7 + 9 + 2$
- ▷ For both cuts, the value of the flow is $|f| = 8$

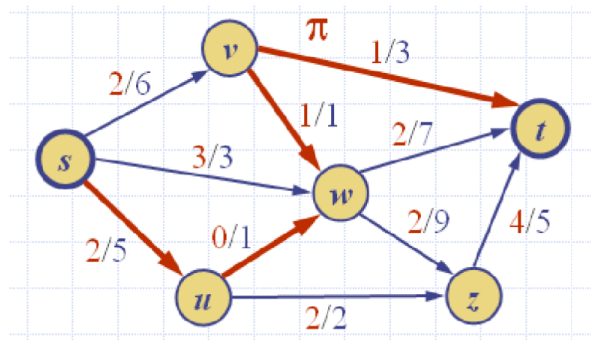
- Lemma1: 对流网络N的流 f , 对任意割 X , $|f|=f(X)$
- Lemma2: 对流网络N的割 X , 对N的任意流 f , 穿过X的流量不会超过X的容量, 即 $f(X) \leq c(X)$
- Theorem1: 任何流的值小于等于任何割的容量, 即对任意流和割 X , 有 $|f| \leq c(X)$
 即最小割的容量=最大流的流量

Augmenting Path 增广路径

考虑网络N的流 f , e 是 u 到 v 的一条边, 定义:

- 边 e 从 u 到 v 的剩余容量 (residual capacity) $\Delta f(u, v) = c(e) - f(e)$ 即剩余容量
- 边 e 从 v 到 u 的剩余容量 (residual capacity) $\Delta f(v, u) = f(e)$ 即本身流量

设 π 是 s 到 t 的一条路径, π 的剩余容量 $\Delta f(\pi)$ 是从 s 到 t 方向上的最小剩余容量; 若 $\Delta f(\pi) > 0$ 则是增广路径。



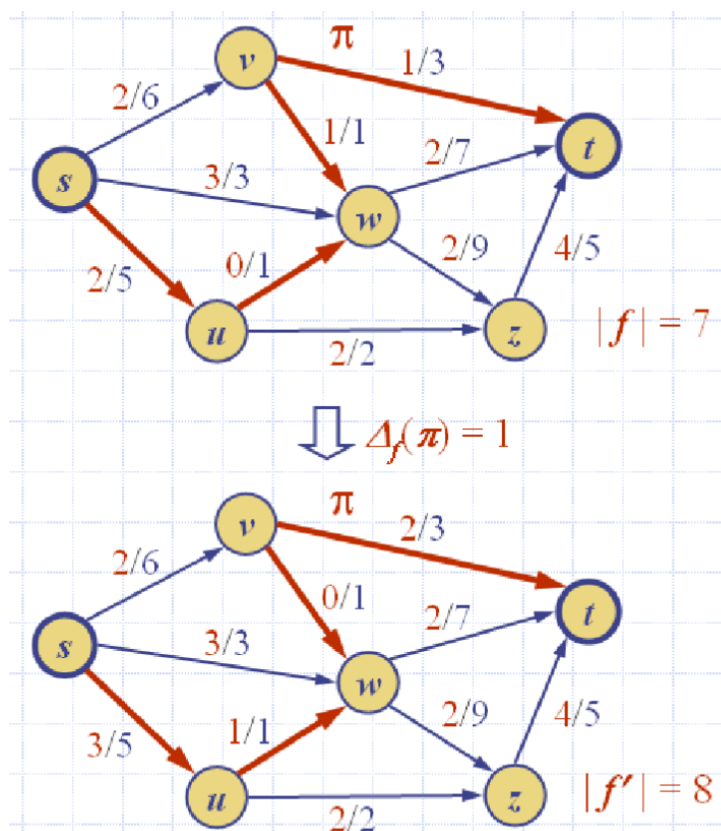
We get the following residual capacities and flow

- ▷ $\Delta_f(s, u) = 3, \Delta_f(u, w) = 1, \Delta_f(w, v) = 1, \Delta_f(v, t) = 2.$
- ▷ $\Delta_f(\pi) = 1$
- ▷ $|f| = 7$

在红色的增广路径 π 上，剩余容量都大于0。

我们总是可以将一条增广路径的剩余容量添加到一个现有流中，从而得到另一个有效流。

- Lemma3: π 是网络N流 f 中的一条增广路径，那么存在N中的流 f' ，使得 $|f'| = |f| + \Delta_f(\pi)$



|| Max-flow, Min-Cut 最大流，最小割

如果网络 N 中的流 f 没有增广路径会怎样？

- Lemma4: 如果网络 N 没有关于流 f 的增广路径，则 f 是最大流 (maximum flow)。同时存在N的割X，满足 $|f| = c(X)$

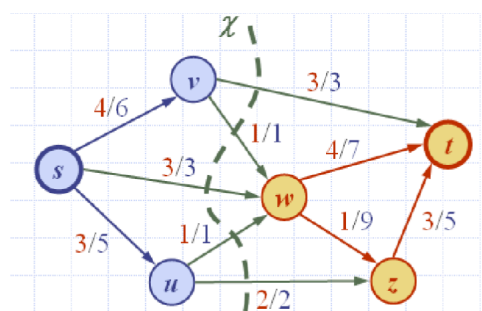
Lemma4和Theorem1给出了最大流最小割定理 (Max-flow, Min-Cut Theorem) , 又名Ford-Fulkerson Theorem

- Theorem2: 最大流的值等于最小割的容量。

定义:

- V_s : 从s通过增广路径可以到达 (邻接) 的顶点集
- V_t : 剩余顶点的集合
- 割 $X = (V_s, V_t)$ 有容量 $c(X) = |f|$ (即Forward edge: $f(e) = c(e)$, Backward edge: $f(e) = 0$)

由此, 流f有最大流量, 且割X有最小容量, 如下图中 $c(X) = |f| = 10$



|| Ford-Fulkerson Theorem

初始, 对每个边e, $f(e)=0$

重复: 直到没有增广路径

搜索增广路径 π

计算瓶颈容量 $\Delta f(\pi) = \min_{e \in \pi} \text{residualCapacity}(e)$

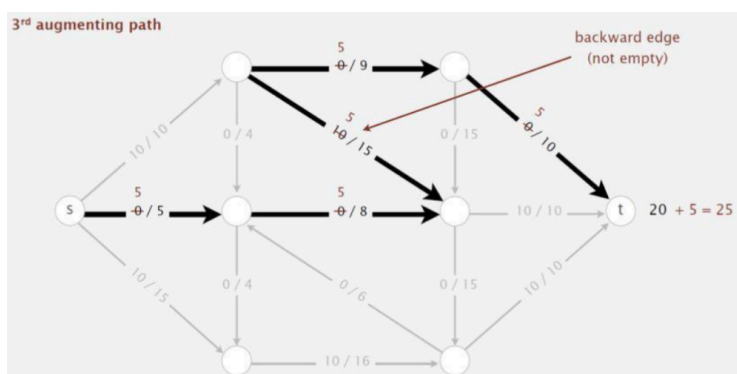
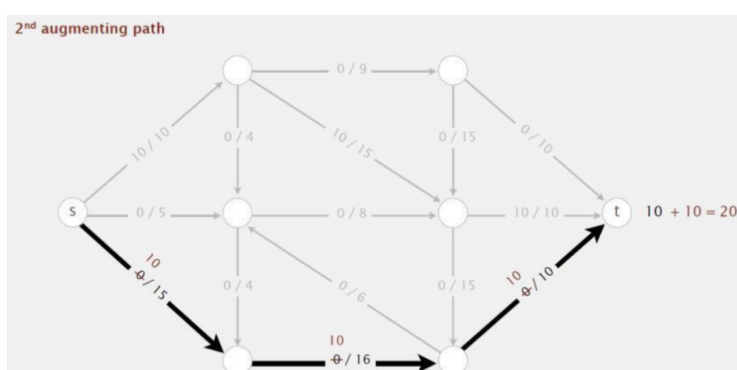
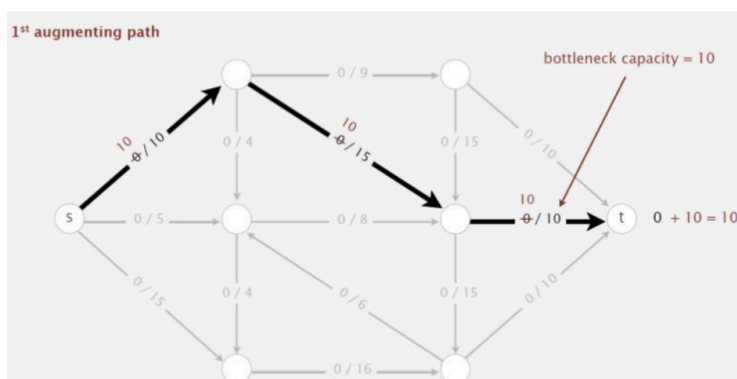
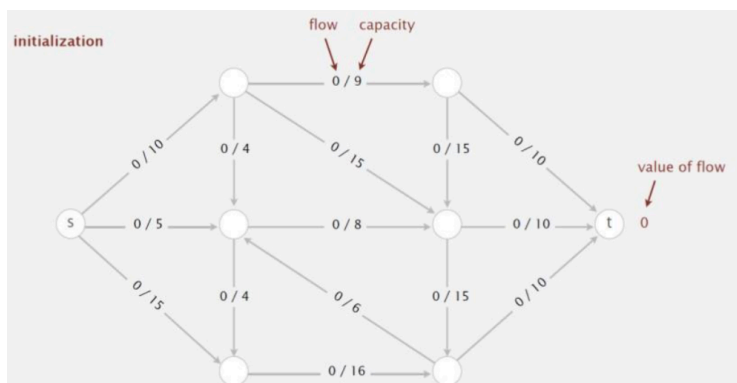
沿着 π 的边增加 $\Delta f(\pi)$, 如果是Forward edge就+ , 如果是Backward edge就- =

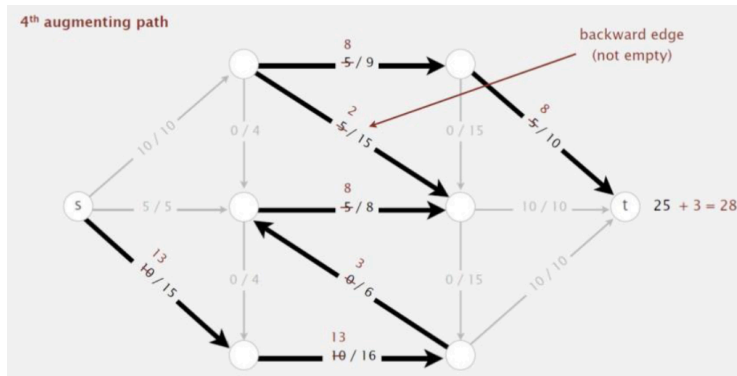
Idea: increase flow along augmenting paths

Augmenting path. Find an undirected path from s to t such that:

- Can increase flow on forward edges (not full).
- Can decrease flow on backward edge (not empty).

这里找的路径, 如果同向没有满即可, 如果逆向不得为0。

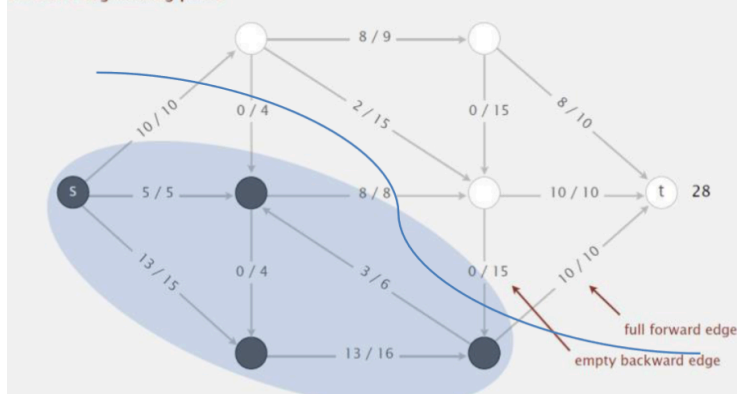




Termination. All paths from s to t are blocked by either a

- Full forward edge.
 - Empty backward edge.
- Cut $\chi = (V_s, V_t)$ has capacity $c(\chi) = |f|$
- ▷ Forward edge: $f(e) = c(e)$
 - ▷ Backward edge: $f(e) = 0$

no more augmenting paths



最后，割线上要么是满的Forward edge，要么是为零的Backward edge。

二分匹配 Bipartite Matching

图 $G=(V,E)$ 被称为二分图 (bipartite) 当且仅当：

- 顶点集 V 可以分为两个不相交的子集 X 和 Y
- G 的每条边在 X 中都有一个端点，在 Y 中有一个端点。（即 X 或 Y 内不存在边）

匹配 (matching) $M \subseteq E$ 是边的子集，其中

- 没有两条边共享同一个顶点
- 一个集合中的每个顶点最多有另一个集合中的一个对应。

暴露/不匹配的 (Exposed/Unmatched) 顶点

- 若顶点 v 没有 M 中的边连接，则称顶点 v 是不匹配的。

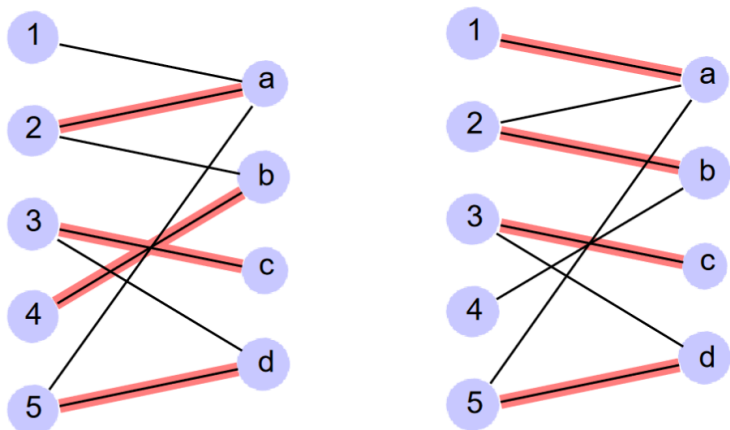
完美匹配 (Perfect Matching)

- 每个顶点都被匹配，不存在暴露的顶点

最大二分匹配问题是找到具有最大边数（在所有匹配中）的匹配 M 。

即：给定一个二分图 $G = (A \cup B, E)$ ，找到一个 $S \subseteq A \times B$ ，它是匹配的，并且尽可能大。

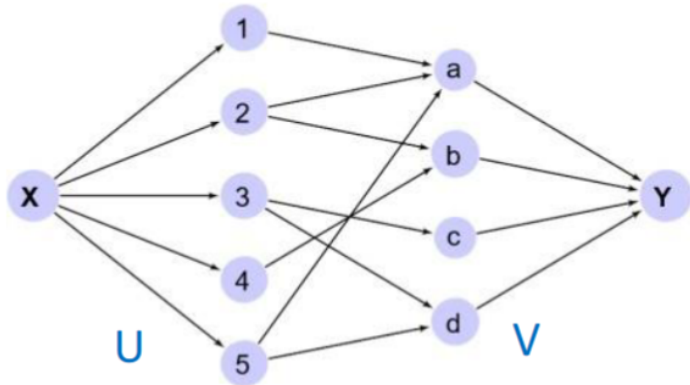
最大匹配的解不一定唯一



使用网络流方法解决匹配问题：

1. 构造具有 source 和 sink 节点的有向图。

引入一个源节点 X 和汇节点 Y ；有 U 集合传入 V 集合，使得 X 传入所有 U 集合， V 集合传入 Y 集合。

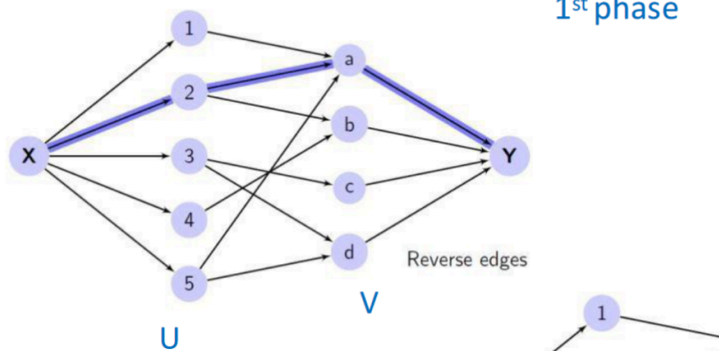


求解未加权有向图的最大流：

2. 使用搜索方法查找从 source 到 sink 的路径。

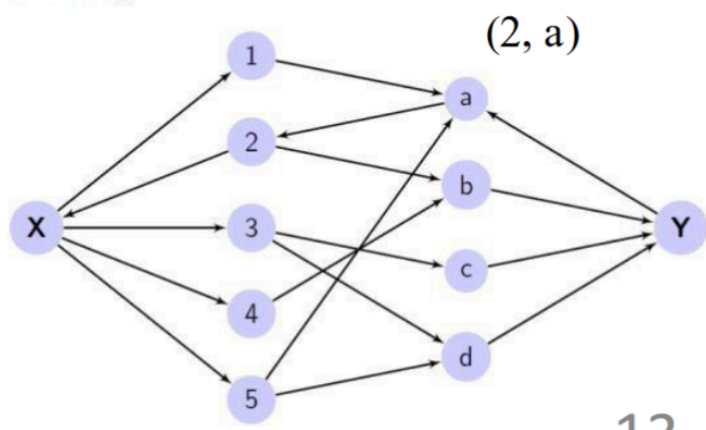
Found path: $X \rightarrow 2 \rightarrow a \rightarrow Y$

1st phase



3. 找到路径后，反转此路径上的每条边。

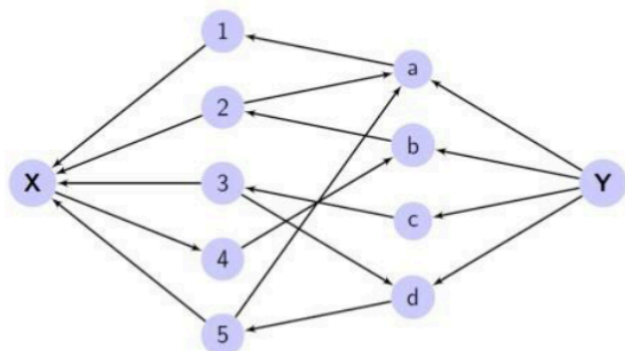
Reverse edges



4. 重复步骤 2 和 3，直到不再存在从 sink 到 source 的路径。

Reverse edges

(1,a),(2,b),(5,d),(3,c)



5. 然后，解是 U 和 V 之间的一组相反的边（即从 V 到 U）。

No more paths found. Matching is reversed edges.

